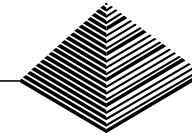
14



Quality Attribute Modeling and Analysis

Do not believe in anything simply because you have heard it . . . Do not believe in anything merely on the authority of your teachers and elders. Do not believe in traditions because they have been handed down for many generations. But after observation and analysis, when you find that anything agrees with reason and is conducive to the good and benefit of one and all, then accept it and live up to it.
—Prince Gautama Siddhartha

In Chapter 2 we listed thirteen reasons why architecture is important, worth studying, and worth practicing. Reason 6 is that the analysis of an architecture enables early prediction of a system's qualities. This is an extraordinarily powerful reason! Without it, we would be reduced to building systems by choosing various structures, implementing the system, measuring the system for its quality attribute responses, and all along the way hoping for the best. Architecture lets us do better than that, much better. We can analyze an architecture to see how the system or systems we build from it will perform with respect to their quality attribute goals, even before a single line of code has been written. This chapter will explore how.

The methods available depend, to a large extent, on the quality attribute to be analyzed. Some quality attributes, especially performance and availability, have well-understood and strongly validated analytic modeling techniques. Other quality attributes, for example security, can be analyzed through checklists. Still others can be analyzed through back-of-the-envelope calculations and thought experiments.

Our topics in this chapter range from the specific, such as creating models and analyzing checklists, to the general, such as how to generate and carry out the thought experiments to perform early (and necessarily crude) analysis. Models

and checklists are focused on particular quality attributes but can aid in the analysis of any system with respect to those attributes. Thought experiments, on the other hand, can consider multiple quality attributes simultaneously but are only applicable to the specific system under consideration.

14.1 Modeling Architectures to Enable Quality Attribute Analysis

Some quality attributes, most notably performance and availability, have well-understood, time-tested analytic models that can be used to assist in an analysis. By *analytic model*, we mean one that supports quantitative analysis. Let us first consider performance.

Analyzing Performance

In Chapter 12 we discussed the fact that models have parameters, which are values you can set to predict values about the entity being modeled (and in Chapter 12 we showed how to use the parameters to help us derive tactics for the quality attribute associated with the model). As an example we showed a queuing model for performance as Figure 12.2, repeated here as Figure 14.1. The parameters of this model are the following:

- The arrival rate of events
- The chosen queuing discipline
- The chosen scheduling algorithm
- The service time for events
- The network topology
- The network bandwidth
- The routing algorithm chosen

In this section, we discuss how such a model can be used to understand the latency characteristics of an architectural design.

To apply this model in an analytical fashion, we also need to have previously made some architecture design decisions. We will use model-view-controller as our example here. MVC, as presented in Section 13.2, says nothing about its deployment. That is, there is no specification of how the model, the view, and the controller are assigned to processes and processors; that's not part of the pattern's concern. These and other design decisions have to be made to transform a pattern into an architecture. Until that happens, one cannot say anything with authority about how an MVC-based implementation will perform. For this example we will assume that there is one instance each of the model, the view, and the controller, and that each instance is allocated to a separate processor. Figure 14.2 shows MVC following this allocation scheme.

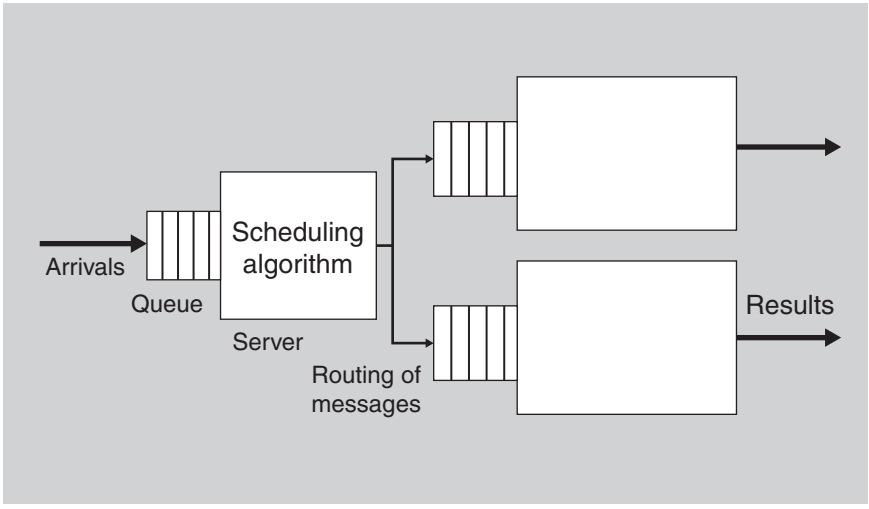


FIGURE 14.1 A queuing model of performance

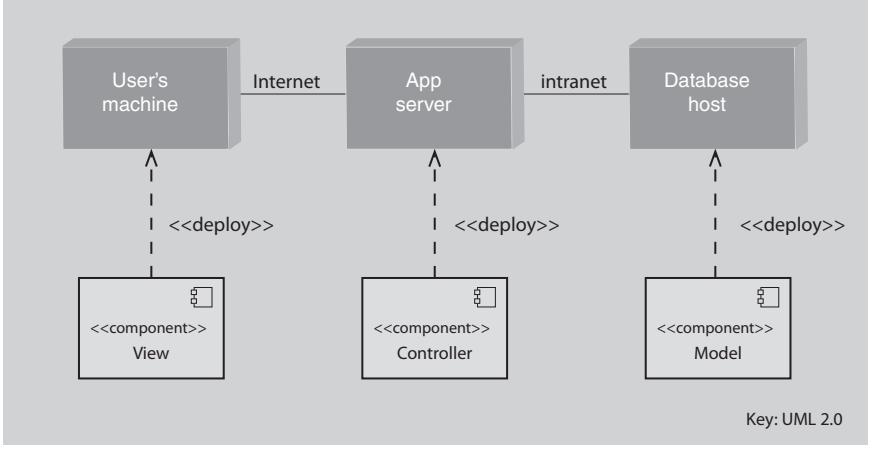


FIGURE 14.2 An allocation view, in UML, of a model-view-controller architecture

Given that quality attribute models such as the performance model shown in Figure 14.1 already exist, the problem becomes how to map these allocation and coordination decisions onto Figure 14.1. Doing this yields Figure 14.3. There are requests coming from users outside the system—labeled as 1 in Figure 14.3—arriving at the view. The view processes the requests and sends some

transformation of the requests on to the controller—labeled as 2. Some actions of the controller are returned to the view—labeled as 3. The controller sends other actions on to the model—labeled 4. The model performs its activities and sends information back to the view—labeled 5.

To analyze the model in Figure 14.3, a number of items need to be known or estimated:

- The frequency of arrivals from outside the system
- The queuing discipline used at the view queue
- The time to process a message within the view
- The number and size of messages that the view sends to the controller
- The bandwidth of the network that connects the view and the controller
- The queuing discipline used by the controller
- The time to process a message within the controller
- The number and size of messages that the controller sends back to the view
- The bandwidth of the network used for messages from the controller to the view
- The number and size of messages that the controller sends to the model
- The queuing discipline used by the model
- The time to process a message within the model
- The number and size of messages the model sends to the view
- The bandwidth of the network connecting the model and the view

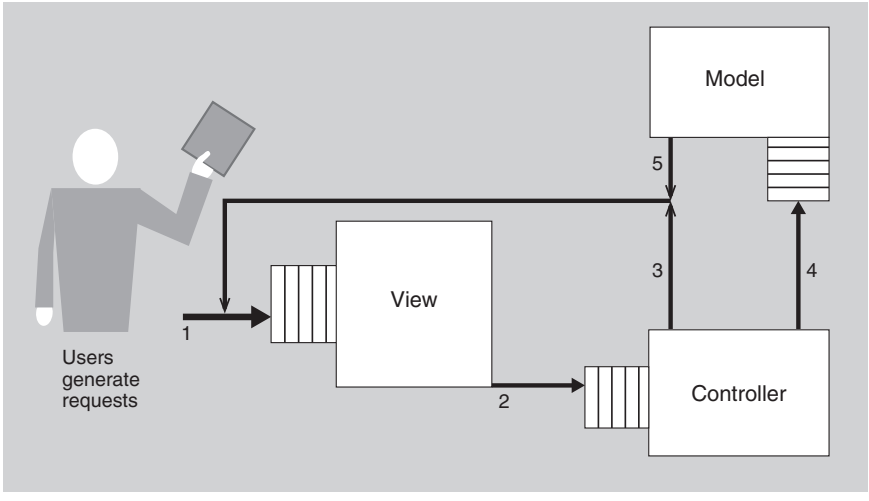


FIGURE 14.3 A queuing model of performance for MVC

Given all of these assumptions, the latency for the system can be estimated. Sometimes well-known formulas from queuing theory apply. For situations where there are no closed-form solutions, estimates can often be obtained through simulation. Simulations can be used to make more-realistic assumptions such as the distribution of the event arrivals. The estimates are only as good as the assumptions, but they can serve to provide rough values that can be used either in design or in evaluation; as better information is obtained, the estimates will improve.

A reasonably large number of parameters must be known or estimated to construct the queuing model shown in Figure 14.3. The model must then be solved or simulated to derive the expected latency. This is the cost side of the cost/benefit of performing a queuing analysis. The benefit side is that as a result of the analysis, there is an estimate for latency, and "what if" questions can be easily answered. The question for you to decide is whether having an estimate of the latency and the ability to answer "what if" questions is worth the cost of performing the analysis. One way to answer this question is to consider the importance of having an estimate for the latency prior to constructing either the system or a prototype that simulates an architecture under an assumed load. If having a small latency is a crucial requirement upon which the success of the system relies, then producing an estimate is appropriate.

Performance is a well-studied quality attribute with roots that extend beyond the computer industry. For example, the queuing model given in Figure 14.1 dates from the 1930s. Queuing theory has been applied to factory floors, to banking queues, and to many other domains. Models for real-time performance, such as rate monotonic analysis, also exist and have sophisticated analysis techniques.

Analyzing Availability

Another quality attribute with a well-understood analytic framework is availability.

Modeling an architecture for availability—or to put it more carefully, modeling an architecture to determine the availability of a system based on that architecture—is a matter of determining the failure rate and the recovery time. As you may recall from Chapter 5, availability can be expressed as

$$\frac{MTBF}{(MTBF + MTTR)}$$

This models what is known as steady-state availability, and it is used to indicate the uptime of a system (or component of a system) over a sufficiently long duration. In the equation, *MTBF* is the *mean time between failure*, which is derived based on the expected value of the implementation's failure probability density function (PDF), and *MTTR* refers to the *mean time to repair*.

Just as for performance, to model an architecture for availability, we need an architecture to analyze. So, suppose we want to increase the availability of a system that uses the broker pattern, by applying redundancy tactics. Figure 14.4

illustrates three well-known redundancy tactics from Chapter 5: *active redundancy*, *passive redundancy*, and *cold spare*. Our goal is to analyze each redundancy option for its availability, to help us choose one.

As you recall, each of these tactics introduces a backup copy of a component that will take over in case the primary component suffers a failure. In our case, a broker replica is employed as the redundant spare. The difference among them is how up to date with current events each backup keeps itself:

- In the case of active redundancy, the active and redundant brokers both receive identical copies of the messages received from the client and server proxies. The internal broker state is synchronously maintained between the active and redundant spare in order to facilitate rapid failover upon detection of a fault in the active broker.
- For the passive redundancy implementation, only the active broker receives and processes messages from the client and server proxies. When using this tactic, checkpoints of internal broker state are periodically transmitted from the active broker process to the redundant spare, using the checkpoint-based rollback tactic.
- Finally, when using the cold spare tactic, only the active broker receives and processes messages from the client and server proxies, because the redundant spare is in a dormant or even powered-off state. Recovery strategies using this tactic involve powering up, booting, and loading the broker implementation on the spare. In this scenario, the internal broker state is rebuilt organically, rather than via synchronous operation or checkpointing, as described for the other two redundancy tactics.

Suppose further that we will detect failure with the *heartbeat* tactic, where each broker (active and spare) periodically transmits a heartbeat message to a separate process responsible for fault detection, correlation, reporting, and recovery. This fault manager process is responsible for coordinating the transition of the active broker role from the failed broker process to the redundant spare.

You can now use the steady state model of availability to assign values for *MTBF* and *MTTR* for each of the three redundancy tactics we are considering. Doing so will be an exercise left to the reader (as you'll see when you reach the discussion questions for this chapter). Because the three tactics differ primarily in how long it takes to bring the backup copy up to speed, *MTTR* will be where the difference among the tactics shows up.

More sophisticated models of availability exist, based on probability. In these models, we can express a probability of failure during a period of time. Given a particular *MTBF* and a time duration *T*, the probability of failure *R* is given by

$$R(T) = e^{\left(\frac{-T}{MTBF}\right)}$$

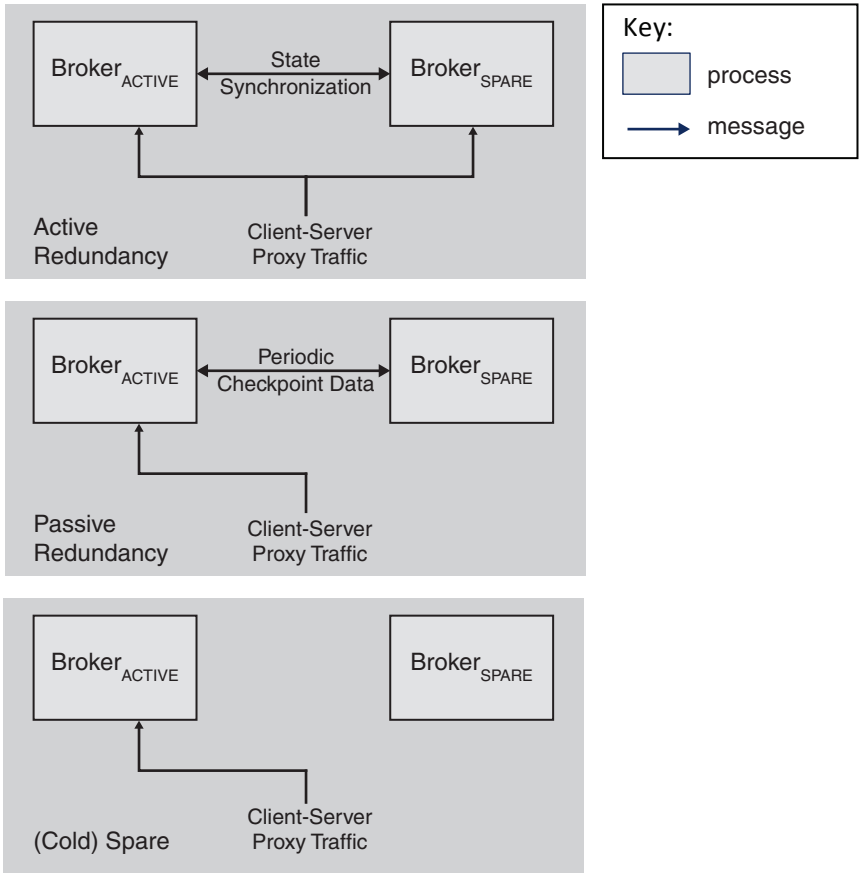


FIGURE 14.4 Redundancy tactics, as applied to a broker pattern

You will recall from Statistics 101 that:

- When two events A and B are independent, the probability that A or B will occur is the sum of the probability of each event: `P(A or B) = P(A) + P(B)`.
- When two events A and B are independent, the probability of both occurring is `P(A and B) = P(A) • P(B)`.
- When two events A and B are dependent, the probability of both occurring is `P(A and B) = P(A) • P(B|A)`, where the last term means "the probability of B occurring, given that A occurs."

We can apply simple probability arithmetic to an architecture pattern for availability to determine the probability of failure of the pattern given the probability of failure of the individual components (and an understanding of their dependency relations). For example, in an architecture pattern employing the passive redundancy tactic, let's assume that the failure of a component (which at any moment might be acting as either the primary or backup copy) is independent of a failure of its counterpart, and that the probability of failure of either is the same. Then the probability that both will fail is `F = (1 - a) **2`, where `a` is the availability of an individual component (assuming that failures are independent).

Still other models take into account different levels of failure severity and degraded operating states of the system. Although the derivation of these formulas is outside the scope of this chapter, you end up with formulas that look like the following for the three redundancy tactics we've been discussing, where the values C2 through C5 are references to the MTBF column of Table 14.1, D2 through D4 refer to the Active column, E2 through E3 refer to the Passive column, and F2 through F3 refer to the Spare column.

- Active redundancy:
  - Availability(MTTR): 1 –((SUM(C2:C5) + D3) × D2)/((C2 × (C2 + C4 + D3) + ((C2 + C4 + D2) × (C3 + C5)) + ((C2 + C4) × (C2 + C4 + D3))))
  - P(Degraded) = ((C3 + C5) × D2)/((C2 × (C2 + C4 + D3) + ((C2 + C4 + D2) × (C3 + C5)) + ((C2 + C4) × (C2 + C4 + D3))))
- Passive redundancy:
  - Availability(MTTR_passive) = 1 – ((SUM(C2:C5) + E3) × E2)/((C2 × (C2 + C4 + E3) + ((C2 + C4 + E2) × (C3 + C5)) + ((C2 + C4) × (C2 + C4 + E3))))
  - P(Degraded) = ((C3 + C5) × E2)/((C2 × (C2 + C4 + E3) + ((C2 + C4 + E2) × (C3 + C5)) + ((C2 + C4) × (C2 + C4 + E3))))
- Spare:
  - Availability(MTTR) = 1 – ((SUM(C2:C5) + F3) × F2)/((C2 × (C2 + C4 + F3) + ((C2 + C4 + F2) × (C3 + C5)) + ((C2 + C4) × (C2 + C4 + F3))))
  - P(Degraded) = ((C3 + C5) × F2)/((C2 × (C2 + C4 + F3) + ((C2 + C4 + F2) × (C3 + C5)) + ((C2 + C4) × (C2 + C4 + F3))))

Plugging in these values for the parameters to the equations listed above results in a table like Table 14.1, which can be easily calculated using any spreadsheet tool. Such a calculation can help in the selection of tactics.

TABLE 14.1 Calculated Availability for an Availability-Enhanced Broker Implementation

Function	Failure Severity	MTBF (Hours)	MTTR (Seconds)		
			Active Redundancy (Hot Spare)	Passive Redundancy (Warm Spare)	Spare (Cold Spare)
Hardware	1	250,000	1	5	900
	2	50,000	30	30	30
Software	1	50,000	1	5	900
	2	10,000	30	30	30
Availability			0.9999998	0.999990	0.9994

The Analytic Model Space

As we discussed in the preceding sections, there are a growing number of analytic models for some aspects of various quality attributes. One of the quests of software engineering is to have a sufficient number of analytic models for a sufficiently large number of quality attributes to enable prediction of the behavior of a designed system based on these analytic models. Table 14.2 shows our current status with respect to this quest for the seven quality attributes discussed in Chapters 5–11.

TABLE 14.2 A Summary of the Analytic Model Space

Quality Attribute	Intellectual Basis	Maturity/Gaps
Availability	Markov models; statistical models	Moderate maturity; mature in the hardware reliability domain, less mature in the software domain. Requires models that speak to state recovery and for which failure percentages can be attributed to software.
Interoperability	Conceptual framework	Low maturity; models require substantial human interpretation and input.
Modifiability	Coupling and cohesion metrics; cost models	Substantial research in academia; still requires more empirical support in real-world environments.
Performance	Queuing theory; real-time scheduling theory	High maturity; requires considerable education and training to use properly.
Security	No architectural models	
Testability	Component interaction metrics	Low maturity; little empirical validation.
Usability	No architectural models	

As the table shows, the field still has a great deal of work to do to achieve the quest for well-validated analytic models to predict behavior, but there is a great deal of activity in this area (see the "For Further Reading" section for additional papers). The remainder of this chapter deals with techniques that can be used *in addition to* analytic models.

14.2 Quality Attribute Checklists

For some quality attributes, checklists exist to enable the architect to test compliance or to guide the architect when making design decisions. Quality attribute checklists can come from industry consortia, from government organizations, or from private organizations. In large organizations they may be developed in house.

These checklists can be specific to one or more quality attributes; checklists for safety, security, and usability are common. Or they may be focused on a particular domain; there are security checklists for the financial industry, industrial process control, and the electric energy sector. They may even focus on some specific aspect of a single quality attribute: cancel for usability, as an example.

For the purposes of certification or regulation, the checklists can be used by auditors as well as by the architect. For example, two of the items on the checklist of the Payment Card Industry (PCI) are to only persist credit card numbers in an encrypted form and to never persist the security code from the back of the credit card. An auditor can ask to examine stored credit card data to determine whether it has been encrypted. The auditor can also examine the schema for data being stored to see whether the security code has been included.

This example reveals that design and analysis are often two sides of the same coin. By considering the kinds of analysis to which a system will be subjected (in this case, an audit), the architect will be led into making important early architectural decisions (making the decisions the auditors will want to find).

Security checklists usually have heavy process components. For example, a security checklist might say that there should be an explicit security policy within an organization, and a cognizant security officer to ensure compliance with the policy. They also have technical components that the architect needs to examine to determine the implications on the architecture of the system being designed or evaluated. For example, the following is an item from a security checklist generated by a group chartered by an organization of electric producers and distributors. It pertains to embedded systems delivering electricity to your home:

> A designated system or systems shall daily or on request obtain current version numbers, installation date, configuration settings, patch level on all elements of a [portion of the electric distribution] system,

In Search of a Grand Unified Theory for Quality Attributes

How do we create analytic models for those quality attribute aspects for which none currently exist? I do not know the answer to this question, but if we had a basis set for quality attributes, we would be in a better position to create and validate quality attribute models. By *basis set* I mean a set of orthogonal concepts that allow one to define the existing set of quality attributes. Currently there is much overlap among quality attributes; a basis set would enable discussion of tradeoffs in terms of a common set of fundamental and possibly quantifiable concepts. Once we have a basis set, we could develop analytic models for each of the elements of the set, and then an analytic model for a particular quality attribute becomes a composition of the models of the portions of the basis set that make up that quality attribute.

What are some of the elements of this basis set? Here are some of my candidates:

- *Time.* Time is the basis for performance, some aspects of availability, and some aspects of usability. Time will surely be one of the fundamental concepts for defining quality attributes.
- *Dependencies among structural elements.* Modifiability, security, availability, and performance depend in some form or another on the strength of connections among various structural elements. Coupling is a form of dependency. Attacks depend on being able to move from one compromised element to a currently uncompromised element through some dependency. Fault propagation depends on dependencies. And one of the key elements of performance analysis is the dependency of one computation on another. Enumeration of the fundamental forms of dependency and their properties will enable better understanding of many quality attributes and their interaction.
- *Access.* How does a system promote or deny access through various mechanisms? Usability is concerned with allowing smooth access for humans; security is concerned with allowing smooth access for some set of requests but denying access to another set of requests. Interoperability is concerned with establishing connections and accessing information. Race conditions, which undermine availability, come about through unmediated access to critical computations.

These are some of my candidates. I am sure there are others. The general problem is to define a set of candidates for the basis set and then show how current definitions of various quality attributes can be recast in terms of the elements of the basis set. I am convinced that this is a problem that needs to be solved prior to making substantial progress in the quest for a rich enough set of analytic models to enable prediction of system behavior across the quality attributes important for a system.

—LB

compare these with inventory and configuration databases, and log all discrepancies.

This kind of rule is intended to detect malware masquerading as legitimate components of a system. The architect will look at this item and conclude the following:

- The embedded portions of the system should be able to report their version number, installation date, configuration settings, and patch levels. One technique for doing this is to use "reflection" for each component in the system. Reflection now becomes one of the important patterns used in this system.
- Each software update or patch should maintain this information. One technique for doing this is to have automated update and patch mechanisms. The architecture could also realize this functionality through reflection.
- A system must be designated to query the embedded components and persist the information. This means
  - There must be overall inventory and configuration databases.
  - Logs of discrepancies between current values and overall inventory must be generated and sent to appropriate recipients.
  - There must be network connections to the embedded components. This affects the network topology.

The creation of quality attribute checklists is usually a time-consuming activity, undertaken by multiple individuals and typically refined and evolved over time. Domain specialists, quality attribute specialists, and architects should all contribute to the development and validation of these checklists.

The architect should treat the items on an applicable checklist as requirements, in that they need to be understood and prioritized. Under particular circumstances, an item in a checklist may not be met, but the architect should have a compelling case as to why it is not.

14.3 Thought Experiments and Back-of-the-Envelope Analysis

A thought experiment is a fancy name for the kinds of discussions that developers and architects have on a daily basis in their offices, in their meetings, over lunch, over whiteboards, in hallways, and around the coffee machine. One of the participants might draw two circles and an arrow on the whiteboard and make an assertion about the quality attribute behavior of these two circles and the arrow in a particular context; a discussion ensues. The discussion can last for a long time, especially if the two circles are augmented with a third and one more arrow, or if some of the assumptions underlying a circle or an arrow are still in flux. In this section, we describe this process somewhat more formally.

The level of formality one would use in performing a thought experiment is, as with most techniques discussed in this book, a question of context. If two people with a shared understanding of the system are performing the thought experiment for their own private purposes, then circles and lines on a whiteboard are adequate, and the discussion proceeds in a kind of shorthand. If a third person is to review the results and the third person does not share the common understanding, then sufficient details must be captured to enable the third person to understand the argument—perhaps a quick legend and a set of properties need to be added to the diagram. If the results are to be included in documentation as design rationale, then even more detail must be captured, as discussed in Chapter 18. Frequently such thought experiments are accompanied by rough analyses—back-of-the-envelope analyses—based on the best data available, based on past experiences, or even based on the guesses of the architects, without too much concern for precision.

The purpose of thought experiments and back-of-the-envelope analysis is to find problems or confirmation of the nonexistence of problems in the quality attribute requirements as applied to sunny-day use cases. That is, for each use case, consider the quality attribute requirements that pertain to that use case and analyze the use case with the quality attribute requirements in mind. Models and checklists focus on one quality attribute. To consider other quality attributes, one must model or have a checklist for the second quality attribute and understand how those models interact. A thought experiment may consider several of the quality attribute requirements simultaneously; typically it will focus on just the most important ones.

The process of creating a thought experiment usually begins with listing the steps associated with carrying out the use case under consideration; perhaps a sequence diagram is employed. At each step of the sequence diagram, the (mental) question is asked: What can go wrong with this step with respect to any of the quality attribute requirements? For example, if the step involves user input, then the possibility of erroneous input must be considered. Also the user may not have been properly authenticated and, even if authenticated, may not be authorized to provide that particular input. If the step involves interaction with another system, then the possibility that the input format will change after some time must be considered. The network passing the input to a processor may fail; the processor performing the step may fail; or the computation to provide the step may fail, take too long, or be dependent on another computation that may have had problems. In addition, the architect must ask about the frequency of the input, and the anticipated distribution of requests (e.g., Are service requests regular and predictable or irregular and "bursty"?), other processes that might be competing for the same resources, and so forth. These questions go on and on.

For each possible problem with respect to a quality attribute requirement, the follow-on questions consist of things like these:

- Are there mechanisms to detect that problem?

- Are there mechanisms to prevent or avoid that problem?
- Are there mechanisms to repair or recover from that problem if it occurs?
- Is this a problem we are willing to live with?

The problems hypothesized are scrutinized in terms of a cost/benefit analysis. That is, what is the cost of preventing this problem compared to the benefits that accrue if the problem does not occur?

As you might have gathered, if the architects are being thorough and if the problems are significant (that is, they present a large risk for the system), then these discussions can continue for a long time. The discussions are a normal portion of design and analysis and will naturally occur, even if only in the mind of a single designer. On the other hand, the time spent performing a particular thought experiment should be bounded. This sounds obvious, but every grey-haired architect can tell you war stories about being stuck in endless meetings, trapped in the purgatory of "analysis paralysis."

Analysis paralysis can be avoided with several techniques:

- "Time boxing": setting a deadline on the length of a discussion.
- Estimating the cost if the problem occurs and not spending more than that cost in the analysis. In other words, do not spend an inordinate amount of time in discussing minor or unlikely potential problems.

Prioritizing the requirements will help both with the cost estimation and with the time estimation.

14.4 Experiments, Simulations, and Prototypes

In many environments it is virtually impossible to do a purely top-down architectural design; there are too many considerations to weigh at once and it is too hard to predict all of the relevant technological barriers. Requirements may change in dramatic ways, or a key assumption may not be met: We have seen cases where a vendor-provided API did not work as specified, or where an API exposing a critical function was simply missing.

Finding the sweet spot within the enormous architectural design space of complex systems is not feasible by reflection and mathematical analysis alone; the models either aren't precise enough to deal with all of the relevant details or are so complicated that they are impractical to analyze with tractable mathematical techniques.

The purpose of *experiments*, *simulations*, and *prototypes* is to provide alternative ways of analyzing the architecture. These techniques are invaluable in

resolving tradeoffs, by helping to turn unknown architectural parameters into constants or ranges. For example, consider just a few of the questions that might occur when creating a web-conferencing system—a distributed client-server infrastructure with real-time constraints:

- Would moving to a distributed database from local flat files negatively impact feedback time (latency) for users?
- How many participants could be hosted by a single conferencing server?
- What is the correct ratio between database servers and conferencing servers?

These sorts of questions are difficult to answer analytically. The answers to these questions rely on the behavior and interaction of third-party components such as commercial databases, and on performance characteristics of software for which no standard analytical models exist. The approach used for the web-conferencing architecture was to build an extensive testing infrastructure that supported simulations, experiments, and prototypes, and use it to compare the performance of each incremental modification to the code base. This allowed the architect to determine the effect of each form of improvement before committing to including it in the final system. The infrastructure includes the following:

- A client simulator that makes it appear as though tens of thousands of clients are simultaneously interacting with a conferencing server.
- Instrumentation to measure load on the conferencing server and database server with differing numbers of clients.

The lesson from this experience is that experimentation can often be a critical precursor to making significant architectural decisions. Experimentation must be built into the development process: building experimental infrastructure can be time-consuming, possibly requiring the development of custom tools. Carrying out the experiments and analyzing their results can require significant time. These costs must be recognized in project schedules.

14.5 Analysis at Different Stages of the Life Cycle

Depending on your project's state of development, different forms of analysis are possible. Each form of analysis comes with its own costs. And there are different levels of confidence associated with each analysis technique. These are summarized in Table 14.3.

TABLE 14.3 Forms of Analysis, Their Life-Cycle Stage, Cost, and Confidence in Their Outputs

Life-Cycle Stage	Form of Analysis	Cost	Confidence
Requirements	Experience-based analogy	Low	Low–High
Requirements	Back-of-the-envelope	Low	Low–Medium
Architecture	Thought experiment	Low	Low–Medium
Architecture	Checklist	Low	Medium
Architecture	Analytic model	Low–Medium	Medium
Architecture	Simulation	Medium	Medium
Architecture	Prototype	Medium	Medium–High
Implementation	Experiment	Medium–High	Medium–High
Fielded System	Instrumentation	Medium–High	High

The table shows that analysis performed later in the life cycle yields results that merit high confidence. However, this confidence comes at a price. First, the cost of performing the analysis also tends to be higher. But the cost of changing the system to fix a problem uncovered by analysis skyrockets later in the life cycle.

Choosing an appropriate form of analysis requires a consideration of all of the factors listed in Table 14.3: What life-cycle stage are you currently in? How important is the achievement of the quality attribute in question and how worried are you about being able to achieve the goals for this attribute? And finally, how much budget and schedule can you afford to allocate to this form of risk mitigation? Each of these considerations will lead you to choose one or more of the analysis techniques described in this chapter.

14.6 Summary

Analysis of an architecture enables early prediction of a system's qualities. We can analyze an architecture to see how the system or systems we build from it will perform with respect to their quality attribute goals. Some quality attributes, most notably performance and availability, have well-understood, time-tested analytic models that can be used to assist in quantitative analysis. Other quality attributes have less sophisticated models that can nevertheless help with predictive analysis.

For some quality attributes, checklists exist to enable the architect to test compliance or to guide the architect when making design decisions. Quality attribute checklists can come from industry consortia, from government organizations, or from private organizations. In large organizations they may be developed in house. The architect should treat the items on an applicable checklist as requirements, in that they need to be understood and prioritized.

Thought experiments and back-of-the-envelope analysis can often quickly help find problems or confirm the nonexistence of problems with respect to quality attribute requirements. A thought experiment may consider several of the quality attribute requirements simultaneously; typically it will focus on just the most important ones. Experiments, simulations, and prototypes allow the exploration of tradeoffs, by helping to turn unknown architectural parameters into constants or ranges whose values may be measured rather than estimated.

Depending on your project's state of development, different forms of analysis are possible. Each form of analysis comes with its own costs and its own level of confidence associated with each analysis technique.

14.7 For Further Reading

There have been many papers and books published describing how to build and analyze architectural models for quality attributes. Here are just a few examples.

Availability

Many availability models have been proposed that operate at the architecture level of analysis. Just a few of these are [Gokhale 05] and [Yacoub 02].

A discussion and comparison of different black-box and white-box models for determining software reliability can be found in [Chandran 10].

A book relating availability to disaster recovery and business recovery is [Schmidt 10].

Interoperability

An overview of interoperability activities can be found in [Brownsword 04].

Modifiability

Modifiability is typically measured through complexity metrics. The classic work on this topic is [Chidamber 94].

More recently, analyses based on design structure matrices have begun to appear [MacCormack 06].

Performance

Two of the classic works on software performance evaluation are [Smith 01] and [Klein 93].

A broad survey of architecture-centric performance evaluation approaches can be found in [Koziolek 10].

Security

Checklists for security have been generated by a variety of groups for different domains. See for example:

- Credit cards, generated by the Payment Card Industry: www.pcisecurity-standards.org/security_standards/
- Information security, generated by the National Institute of Standards and Technology (NIST): [NIST 09].
- Electric grid, generated by Advanced Security Acceleration Project for the Smart Grid: www.smartgridipedia.org/index.php/ASAP-SG
- Common Criteria. An international standard (ISO/IEC 15408) for computer security certification: www.commoncriteriaportal.org

Testability

Work in measuring testability from an architectural perspective includes measuring testability as the measured complexity of a class dependency graph derived from UML class diagrams, and identifying class diagrams that can lead to code that is difficult to test [Baudry 05]; and measuring controllability and observability as a function of data flow [Le Traon 97].

Usability

A checklist for usability can be found at www.stcsig.org/usability/topics/articles/he-checklist.html

Safety

A checklist for safety is called the Safety Integrity Level: en.wikipedia.org/wiki/Safety_Integrity_Level

Applications of Modeling and Analysis

For a detailed discussion of a case where quality attribute modeling and analysis played a large role in determining the architecture as it evolved through a number of releases, see [Graham 07].

14.8 Discussion Questions

1. Build a spreadsheet for the steady-state availability equation *MTBF* / (*MTBF* + *MTTR*). Plug in different but reasonable values for *MTBF* and *MTTR* for each of the active redundancy, passive redundancy, and cold spare tactics. Try values for *MTBF* that are very large compared to *MTTR*, and also try values for *MTBF* that are much closer in size to *MTTR*. What do these tell you about which tactics you might want to choose for availability?
2. Enumerate as many responsibilities as you can that need to be carried out for providing a "cancel" operation in a user interface. *Hint:* There are at least 21 of them, as indicated in a publication by (*strong hint!*) one of the authors of this book whose last name (*unbelievably strong hint!*) begins with "B."
3. The M/M/1 (look it up!) queuing model has been employed in computing systems for decades. Where in your favorite computing system would this model be appropriate to use to predict latency?
4. Suppose an architect produced Figure 14.5 while you were sitting watching him. Using thought experiments, how can you determine the performance and availability of this system? What assumptions are you making and what conclusions can you draw? How definite are your conclusions?

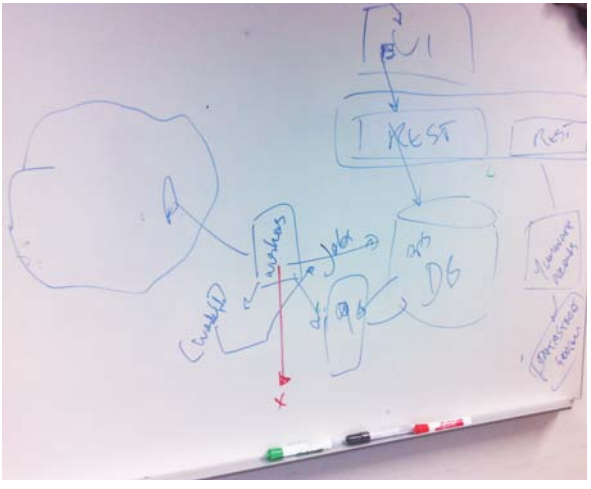


FIGURE 14.5 Capture of a whiteboard sketch from an architect

This page intentionally left blank

PART THREE

ARCHITECTURE IN THE LIFE CYCLE

Part I of this book introduced architecture and the various contextual lenses through which it could be viewed. To recap from Chapter 3, those contexts include the following:

- *Technical*. What technical role does the software architecture play in the system or systems of which it's a part? Part of the answer to this is what Part II of our book is about and the rest is included in Part IV. Part II describes how decisions are made, and Part IV describes the environment that determines whether the results of the decisions satisfy the needs of the organization.
- *Project life cycle*. How does a software architecture relate to the other phases of a software development life cycle? The answer to this is what Part III of our book is about.
- *Business*. How does the presence of a software architecture affect an organization's business environment? The answer to this is what Part IV of our book is about.
- *Professional*. What is the role of a software architect in an organization or a development project? The answer to this is threaded throughout the entire book, but especially in Chapter 24, where we treat the duties, skills, and knowledge of software architects.

Part II concentrated on the *technical* context of software architecture. In our philosophy, this is tantamount to understanding quality attributes. If you have a deep understanding of how architecture affects quality attributes, then you have mastered most of what you need to know about making design decisions.

Here in Part III we turn our attention to how to constructively apply that knowledge within the context of a particular software development project. Here is where software architecture meets software engineering: How do architecture concerns affect the gathering of requirements, the carrying out of design decisions, the validation and capturing of the design, and the transformation of design into implementation? In Part III, we'll find out.

A Word about Methods

Because this is a book about software architecture in *practice*, we've tried to spell out specific methods in enough detail so that you can emulate them. You'll see PALM, a method for eliciting business goals that an architecture should accommodate. You'll see Views and Beyond, an approach for documenting architecture in a set of views that serve stakeholders and their concerns. You'll see ATAM, a method for evaluating an architecture against stakeholders' ideas of what quality attributes it should provide. You'll see CBAM, a method for assessing which evolutionary path of an architecture will best serve stakeholders' needs.

All of these methods rely in some way or another on tapping stakeholders' knowledge about what an architecture under development should provide. As presented in their respective chapters, each of these methods includes a similar process of identifying the relevant stakeholders, putting them in a room together, presenting a briefing about the method that the stakeholders have been assembled to participate in, and then launching into the method.

So why is it necessary to put all of the stakeholders in the same room? The short answer is that it isn't. There are (at least) three major engagement models for conducting an architecture-focused method. Why three? Because we have identified two important factors, each of which has two values, that describe four potential engagement models for gathering information from stakeholders. These two factors are

1. Location (co-located or distributed)
2. Synchronicity (synchronous or asynchronous)

One option (co-located and asynchronous) makes no sense, and so we are left with three *viable* engagement models. The advantages and disadvantages we've observed of each engagement model follow.

Why has the big-meeting format (co-located, synchronous) tended to prevail? There are several reasons:

- It compresses the time required for the method. Time on site for remote participants is minimized, although as we will see, travel time is not considered in this argument. All of the stakeholders are available with minimal external distractions.
- It emphasizes the importance of the method. Any meeting important enough to bring multiple people together for an extended time must be judged by management to be important.
- It benefits from the helpful group mentality that emerges when people are in the same room working toward a common goal. The group mentality fosters buy-in to the architecture and buy-in to the reasons it exists. Putting stakeholders in the same room lets them open communication paths with the architect and with each other, paths that will often remain open long

Model	Advantages	Disadvantages
All stakeholders in the same room for the duration of the exercise (co-located and synchronous).	All stakeholders participate equally. Group mentality produces buy-in for architecture and the results of the exercise. Enduring communication paths are opened among stakeholders. This option takes the shortest calendar time.	Scheduling can be problematic. Some stakeholders might not be forthcoming in a crowd. Stakeholders might incur substantial travel costs to attend.
Some stakeholders participate in exercise remotely (distributed and synchronous).	Saves travel costs for remote participants; this option might permit participation by stakeholders who otherwise would not be able to contribute.	Technology is a limiting factor; remote participants almost always are second-class citizens in terms of their participation and after-exercise "connection" to other participants.
Facilitators interviewing stakeholders individually or in small groups (distributed and asynchronous).	Allows for in-depth interaction between facilitators and stakeholders. Eliminates group factors that might inhibit a stakeholder from speaking in public.	If stakeholders are widely distributed, increased travel costs incurred by facilitator(s). Reduced group buy-in. Reduced group mentality. Reduced after-exercise communication among stakeholders. Exercise stretched out over a longer period of calendar time.

after the meeting has run its course. We always enjoy seeing business cards exchanged with handshakes when stakeholders meet each other for the first time. Putting the architect in a room full of stakeholders for a couple of days is a very healthy thing for any project.

But there are, as ever, tradeoffs. The big-meeting format can be costly and difficult to fit into an already crowded project schedule. Often the hardest aspect of executing any of our methods is finding two contiguous days when all the important stakeholders are available. Also, the travel costs associated with a big meeting can be substantial in a distributed organization. And some stakeholders might not be as forthcoming as we would like if they are in a room surrounded by strong-willed peers or higher-ups (although our methods use facilitation techniques to try to correct for this).

So which model is best? You already know the answer: It depends. You can see the tradeoffs among the different approaches. Pick the one that does the best job for your organization and its particularities.

Conclusion

As you read Part III and learn about architecture methods, remember that the form of the method we present is the one in which the most practical experience resides. But:

1. You can always adjust the engagement model to be something other than everybody-in-the-same-room if that will work better for you.
2. Whereas the steps of a method are nominally carried out in sequential order according to a set agenda, sometimes there must be dynamic modifications to the schedule to accommodate personnel availability or architectural information. Every situation is unique, and there may be times when you need to return briefly to an earlier step, jump forward to a later step, or iterate among steps, as the need dictates.

P.S.: We do provide one example of a shortened version of one of our methods —the ATAM. We call this *Lightweight Architecture Evaluation*, and it is described in Chapter 21.